📚 LibraFlow — Guide de réalisation

Projet Laravel 12 — Gestion de bibliothèque scolaire numérique

IAI-Togo | Outils de Programmation Web

Phase	Contenu	Durée estimée
1	Installation & Configuration	30 min
2	Migrations & Modèles Eloquent	1h
3	Rôles & Policies	45 min
4	Controllers & Logique métier	1h30
5	Vues Blade & Interface	1h30
6	Finalisation & Soutenance	1h

Phase 1 — Installation & Configuration

> **Objectif**
> Créer le projet Laravel, configurer la base de données MySQL avec XAMPP,
> installer Laravel Breeze pour l'authentification et préparer le modèle User avec les rôles.

Étape 1 — Vérifier que tout est prêt

Ouvre CMD et vérifie que ces outils sont installés :

```
php -v
composer -v
node -v
npm -v
```

> **Résultats attendus**
> • php -v → PHP 8.2.x ou supérieur
> • composer -v → Composer version 2.x
> • node -v → Node.js v18 ou supérieur
> • npm -v → npm 9.x ou supérieur

Étape 2 — Créer le projet Laravel

Dans CMD, navigue vers le dossier htdocs de XAMPP :

```
cd C:\xampp\htdocs
composer create-project laravel/laravel libraflow
cd libraflow
```

Étape 3 — Créer la base de données

1. Ouvre XAMPP Control Panel → clique Start sur Apache et MySQL
2. Va sur http://localhost/phpmyadmin
3. Clique sur Nouvelle base de données (à gauche)
4. Nom : libraflow → clique Créer

Étape 4 — Configurer le fichier .env

Ouvre .env avec VS Code et modifie ces lignes :

```
APP_NAME=LibraFlow
APP_URL=http://localhost/libraflow/public

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
```

```
DB_DATABASE=libraflow
DB_USERNAME=root
DB_PASSWORD=
```

Étape 5 — Ajouter la colonne role

Ouvre database/migrations/xxxx_create_users_table.php et ajoute :

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    // AJOUTE CETTE LIGNE :
    $table->enum('role', ['admin','bibliothecaire','lecteur'])
          ->default('lecteur');
    $table->rememberToken();
    $table->timestamps();
});
```

Étape 6 — Installer Laravel Breeze

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
npm run build
```

Étape 7 — Migrations & modèle User

```
php artisan migrate
```

Ouvre app/Models/User.php et ajoute :

```
protected $fillable = ['name', 'email', 'password', 'role'];


public function isAdmin(): bool { return $this->role === 'admin'; }
public function isBibliothecaire(): bool { return $this->role ===
'bibliothecaire'; }
public function isLecteur(): bool { return $this->role === 'lecteur'; }
```

Étape 8 — Lancer le serveur

```
php artisan serve
```

Ouvre http://127.0.0.1:8000 → tu dois voir la page d'accueil Laravel.

✅ Tests Phase 1

Test	Action	Résultat attendu
1	Ouvrir http://127.0.0.1:8000	Page d'accueil Laravel avec Login/Register
2	Cliquer Register et créer un compte	Compte créé, redirigé vers /dashboard
3	phpMyAdmin → table users	Colonne role présente avec valeur lecteur

🗀 Commit Git — Phase 1

```
git init
git add .
git commit -m "Initial commit - Phase 1 : installation Laravel + Breeze + role user"
git branch -M main
git remote add origin https://github.com/ton-compte/libraflow.git
git push -u origin main
```

Résumé Phase 1

Fichier	Ce qu'on a fait
.env	Connexion à la base MySQL de XAMPP
create_users_table.php	Ajout de la colonne role (enum)
User.php	role dans $fillable + 3 méthodes helper

Phase 2 — Migrations & Modèles Eloquent

> **Objectif**
> Créer les 4 tables (books, loans, reservations, settings), leurs modèles Eloquent avec relations et scopes, l'Observer pour gérer available_copies automatiquement, et les paramètres par défaut via SettingSeeder.

Étape 1 — Créer les migrations

```
php artisan make:migration create_books_table
php artisan make:migration create_loans_table
php artisan make:migration create_reservations_table
php artisan make:migration create_settings_table
```

Étape 2 — Migration books

```
Schema::create('books', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('author');
    $table->string('isbn')->unique()->nullable();
    $table->string('category');
    $table->unsignedInteger('total_copies')->default(1);
    $table->unsignedInteger('available_copies')->default(1);
    $table->string('cover_image')->nullable();
    $table->timestamps();
});
```

Étape 3 — Migration loans

```
Schema::create('loans', function (Blueprint $table) {
    $table->id();
    $table->foreignId('book_id')->constrained()->onDelete('cascade');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->timestamp('borrowed_at');
    $table->timestamp('due_at');
    $table->timestamp('returned_at')->nullable();
    $table->decimal('penalty_amount', 8, 2)->default(0);
    $table->timestamps();
});
```

Étape 4 — Migrations reservations & settings

```
// reservations
```

```
Schema::create('reservations', function (Blueprint $table) {
    $table->id();
    $table->foreignId('book_id')->constrained()->onDelete('cascade');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->timestamp('reserved_at');
    $table->enum('status',['en_attente','notifie','annulee'])-
>default('en_attente');
    $table->timestamps();
});

// settings
Schema::create('settings', function (Blueprint $table) {
    $table->id();
    $table->string('key')->unique();
    $table->string('value');
    $table->timestamps();
});
```

Étape 5 — Lancer les migrations

```
php artisan migrate
```

Étape 6 — Créer les modèles

```
php artisan make:model Book
php artisan make:model Loan
php artisan make:model Reservation
php artisan make:model Setting
```

Modèle Book — scopes search() et available()

```
protected $fillable = ['title','author','isbn','category',
                        'total_copies','available_copies','cover_image'];

public function loans() { return $this->hasMany(Loan::class); }
public function reservations() { return $this->hasMany(Reservation::class); }

public function scopeAvailable($query) {
    return $query->where('available_copies', '>', 0);
}
public function scopeSearch($query, $search) {
    return $query->where('title', 'like', "%{$search}%")
                 ->orWhere('author', 'like', "%{$search}%")
                 ->orWhere('category', 'like', "%{$search}%");
}
```

Modèle Loan — scope overdue() et calculatePenalty()

```
protected $casts = [
    'borrowed_at' => 'datetime',
    'due_at'      => 'datetime',
    'returned_at'=> 'datetime',
];

public function scopeOverdue($query) {
    return $query->where('due_at', '<', Carbon::now())
                 ->whereNull('returned_at');
}

public function calculatePenalty(): float {
    if ($this->returned_at || Carbon::now()->lte($this->due_at)) return 0;
    $dailyRate = (float) Setting::getValue('penalite_journaliere', 100);
    return Carbon::now()->diffInDays($this->due_at) * $dailyRate;
}
```

Étape 7 — Observer LoanObserver

```
php artisan make:observer LoanObserver --model=Loan
```

```
// app/Observers/LoanObserver.php
public function created(Loan $loan): void {
    $loan->book->decrement('available_copies'); // emprunt → -1 dispo
}
public function updated(Loan $loan): void {
    if ($loan->isDirty('returned_at') && $loan->returned_at !== null) {
        $loan->book->increment('available_copies'); // retour → +1 dispo
    }
}
```

Enregistre l'Observer dans AppServiceProvider.php :

```
public function boot(): void {
    Loan::observe(LoanObserver::class);
}
```

Étape 8 — SettingSeeder

```
php artisan make:seeder SettingSeeder

// Dans run() :
Setting::setValue('duree_emprunt', '14');
Setting::setValue('penalite_journaliere', '100');
```

```
php artisan db:seed --class=SettingSeeder
```

✅ Tests Phase 2

Test	Commande / Action	Résultat attendu
1	phpMyAdmin → base libraflow	4 nouvelles tables + table settings avec 2 lignes
2	Tinker : Book::create([...])	Livre créé sans erreur
3	Tinker : Loan::create([...]) + $book->fresh()->available_copies	available_copies décrémenté de 1
4	Tinker : Setting::getValue("duree_emprunt")	Retourne "14"

🗁 Commit Git — Branche phase-2

```
git checkout -b phase-2
git add database/migrations/ app/Models/ app/Observers/
git add app/Providers/AppServiceProvider.php database/seeders/
git commit -m "Phase 2 : migrations + modèles Eloquent + Observer + SettingSeeder"
git push origin phase-2
```

Résumé Phase 2

Fichier	Rôle
create_books_table.php	Structure de la table livres
create_loans_table.php	Emprunts avec clés étrangères
create_reservations_table.php	File d'attente réservations
create_settings_table.php	Paramètres configurables
Book.php	Modèle + scopes available() et search()
Loan.php	Modèle + scope overdue() + calculatePenalty()
Reservation.php	Modèle + scope enAttente()
Setting.php	Modèle + getValue() / setValue()
LoanObserver.php	Décrémente/incrémente available_copies auto
SettingSeeder.php	Valeurs par défaut (14 jours, 100 FCFA/jour)

Phase 3 — Rôles & Policies

> **Objectif**
> Sécuriser l'application avec un Middleware CheckRole, des Policies Laravel (BookPolicy, LoanPolicy, ReservationPolicy), et une Gate globale pour l'admin.
> Créer les 3 comptes de test et protéger les routes par rôle.

Étape 1 — Middleware CheckRole

```
php artisan make:middleware CheckRole
```

```
// app/Http/Middleware/CheckRole.php
public function handle(Request $request, Closure $next, string ...$roles):
Response
{
    if (!auth()->check()) return redirect()->route('login');

    if (!in_array(auth()->user()->role, $roles)) {
        abort(403, 'Accès refusé.');
    }
    return $next($request);
}
```

Enregistre l'alias dans bootstrap/app.php :

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\CheckRole::class,
    ]);
})
```

Étape 2 — Créer les Policies

```
php artisan make:policy BookPolicy --model=Book
php artisan make:policy LoanPolicy --model=Loan
php artisan make:policy ReservationPolicy --model=Reservation
```

BookPolicy — qui peut gérer les livres ?

```
public function create(User $user): bool {
    return in_array($user->role, ['admin', 'bibliothecaire']);
}
public function update(User $user, Book $book): bool {
    return in_array($user->role, ['admin', 'bibliothecaire']);
}
public function delete(User $user, Book $book): bool {
```

```
    return $user->role === 'admin'; // seul l'admin peut supprimer
}
```

LoanPolicy — qui peut gérer les emprunts ?

```
public function view(User $user, Loan $loan): bool {
    if ($user->isLecteur()) return $loan->user_id === $user->id;
    return true;
}
public function create(User $user): bool {
    return in_array($user->role, ['admin', 'bibliothecaire']);
}
```

Étape 3 — Gate admin dans AppServiceProvider

```
Gate::before(function (User $user, string $ability) {
    if ($user->isAdmin()) return true; // l'admin bypasse tout
});
```

> **Pourquoi Gate::before ?**
> Plutôt que de vérifier isAdmin() dans chaque Policy, une seule règle globale court-circuite toutes les vérifications. C'est le principe DRY (Don't Repeat Yourself). Très bon point à mentionner en soutenance !

Étape 4 — Routes protégées (web.php)

```
// Accessible à tous les connectés
Route::middleware(['auth'])->group(function () {
    Route::get('/books', [BookController::class, 'index'])->name('books.index');
    Route::get('/mon-espace', [LoanController::class, 'monEspace'])
        ->name('lecteur.espace');
});

// Admin + Bibliothécaire seulement
Route::middleware(['auth', 'role:admin,bibliothecaire'])->group(function () {
    Route::resource('books', BookController::class)->except(['index','show']);
    Route::resource('loans', LoanController::class);
    Route::patch('/loans/{loan}/retour', [LoanController::class, 'retour'])
        ->name('loans.retour');
    Route::get('/retards', [LoanController::class, 'retards'])
        ->name('loans.retards');
});

// Admin seulement
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('/settings', [SettingController::class, 'index'])
```

```
        ->name('settings.index');
    Route::put('/settings/{key}', [SettingController::class, 'update'])
        ->name('settings.update');
});
```

Étape 5 — UserSeeder (3 comptes de test)

```
php artisan make:seeder UserSeeder

// Dans run() :
User::create(['name'=>'Super Admin', 'email'=>'admin@libraflow.com',
              'password'=>Hash::make('password'), 'role'=>'admin']);

User::create(['name'=>'Marie Biblio', 'email'=>'biblio@libraflow.com',
              'password'=>Hash::make('password'), 'role'=>'bibliothecaire']);

User::create(['name'=>'Jean Lecteur', 'email'=>'lecteur@libraflow.com',
              'password'=>Hash::make('password'), 'role'=>'lecteur']);

php artisan db:seed --class=UserSeeder
```

✅ Tests Phase 3

Test	Action	Résultat attendu
1	phpMyAdmin → table users	3 comptes avec rôles admin/bibliothecaire/lecteur
2	Accès /settings sans connexion	Redirigé vers /login
3	Connexion lecteur → /settings	Erreur 403 Accès refusé
4	Connexion admin → /settings	Pas de 403 (page 404 normale, controller pas encore fait)
5	Tinker : BookPolicy->update($lecteur,$book)	Retourne false

🗀 Commit Git — Branche phase-3

```
git checkout -b phase-3
git add app/Http/Middleware/CheckRole.php
git add app/Policies/ bootstrap/app.php routes/web.php
git add app/Providers/AppServiceProvider.php database/seeders/UserSeeder.php
git commit -m "Phase 3 : Middleware CheckRole + Policies + Gate admin + routes protégées"
git push origin phase-3
```

Résumé Phase 3

Fichier	Rôle
CheckRole.php	Middleware qui bloque selon le rôle
bootstrap/app.php	Alias "role" pour les routes
BookPolicy.php	Qui peut créer/modifier/supprimer un livre
LoanPolicy.php	Qui peut gérer les emprunts
ReservationPolicy.php	Qui peut gérer les réservations
AppServiceProvider.php	Policies + Gate admin bypass
web.php	Routes groupées par rôle
UserSeeder.php	3 comptes de test (admin, biblio, lecteur)

Phase 4 — Controllers & Logique métier

> **Objectif**
> Créer les 4 controllers (Book, Loan, Reservation, Setting) avec toute la logique métier : emprunts, retours, calcul automatique des pénalités avec Carbon, file d'attente de réservations et notification Mailable au retour d'un livre.

Étape 1 — Créer les controllers

```
php artisan make:controller BookController --resource
php artisan make:controller LoanController --resource
php artisan make:controller ReservationController --resource
php artisan make:controller SettingController --resource
```

Étape 2 — BookController (points clés)

index() — recherche filtrée

```
public function index(Request $request) {
    $query = Book::query();
    if ($request->filled('search'))   $query->search($request->search);
    if ($request->filled('category')) $query->where('category', $request-
>category);
    if ($request->filled('disponible')) $query->available();
    $books = $query->orderBy('title')->paginate(12);
    $categories = Book::select('category')->distinct()->pluck('category');
    return view('books.index', compact('books', 'categories'));
}
```

store() — validation + available_copies automatique

```
public function store(Request $request) {
    $this->authorize('create', Book::class);
    $validated = $request->validate([
        'title'        => 'required|string|max:255',
        'author'       => 'required|string|max:255',
        'total_copies' => 'required|integer|min:1',
        'cover_image'  => 'nullable|image|max:2048',
    ]);
    $validated['available_copies'] = $validated['total_copies']; // = total à la
création
    Book::create($validated);
    return redirect()->route('books.index')->with('success', 'Livre ajouté.');
}
```

Étape 3 — LoanController (points clés)

store() — emprunter un livre avec Carbon

```
public function store(Request $request) {
    $this->authorize('create', Loan::class);
    $book = Book::findOrFail($request->book_id);

    if ($book->available_copies <= 0) {
        return back()->withErrors(['book_id' => 'Plus d\'exemplaires
disponibles.']);
    }
    $duree = (int) Setting::getValue('duree_emprunt', 14);

    Loan::create([
        'book_id'     => $request->book_id,
        'user_id'     => $request->user_id,
        'borrowed_at'=> Carbon::now(),
        'due_at'     => Carbon::now()->addDays($duree), // Carbon ici !
    ]);
    return redirect()->route('loans.index')->with('success', 'Emprunt
enregistré.');
}
```

retour() — retour + pénalité + notification

```
public function retour(Loan $loan) {
    $penalite = $loan->calculatePenalty(); // calcul avant update

    $loan->update([
        'returned_at'    => Carbon::now(),
        'penalty_amount'=> $penalite, // Observer incrémente available_copies
    ]);

    // Notifie le 1er en file d'attente
    $prochain = $loan->book->reservations()->enAttente()
                          ->orderBy('reserved_at')->first();
    if ($prochain) {
        $prochain->update(['status' => 'notifie']);
        Mail::to($prochain->user->email)
            ->send(new ReservationDisponibleMail($loan->book, $prochain->user));
    }
    return redirect()->route('loans.index')->with('success', 'Retour
enregistré.');
}
```

Étape 4 — ReservationController (points clés)

```
public function store(Request $request) {
    $book = Book::findOrFail($request->book_id);

    if ($book->available_copies > 0) {
        return back()->withErrors(['book_id' => 'Livre dispo, empruntez-le
directement !']);
    }
    // Vérifie doublon
    $dejaReserve = Reservation::where('book_id', $book->id)
        ->where('user_id', auth()->id())->where('status','en_attente')->exists();

    if ($dejaReserve) return back()->withErrors(['book_id' => 'Déjà réservé.']);

    $position = Reservation::where('book_id',$book->id)
                        ->where('status','en_attente')->count() + 1;
    Reservation::create([...]);
    return back()->with('success', "Vous êtes {$position}e dans la file.");
}
```

Étape 5 — Mailable ReservationDisponibleMail

```
php artisan make:mail ReservationDisponibleMail --markdown=emails.reservation-
disponible

// Dans .env, pour tester sans vrai email :
MAIL_MAILER=log
MAIL_FROM_ADDRESS="noreply@libraflow.com"
```

Les emails s'écrivent dans storage/logs/laravel.log — pratique pour les tests.

✅ Tests Phase 4

Test	Action	Résultat attendu
1	Connexion biblio → /books/create	Formulaire visible (pas de 403)
2	Connexion lecteur → /books/create	Erreur 403
3	Connexion admin → /settings	Page paramètres visible
4	Tinker : créer emprunt + vérifier available_copies	Décrémenté de 1
5	Tinker : calculatePenalty() sur emprunt 6j de retard	Retourne 600.0

🗁 Commit Git — Branche phase-4

```
git checkout -b phase-4
git add app/Http/Controllers/ app/Mail/
git add resources/views/emails/
git commit -m "Phase 4 : 4 controllers + Mailable ReservationDisponible"
git push origin phase-4
```

Résumé Phase 4

Fichier	Rôle
BookController.php	CRUD livres + recherche + filtres
LoanController.php	Emprunts, retours, pénalités Carbon, mon espace
ReservationController.php	File d'attente + position
SettingController.php	Paramètres admin
ReservationDisponibleMail.php	Email automatique au retour d'un livre

Phase 5 — Vues Blade & Interface

> **Objectif**
> Créer les 10 vues Blade avec Tailwind CSS : layout principal, catalogue livres, fiche livre avec disponibilité visuelle, dashboard bibliothécaire, espace lecteur avec pénalités bienveillantes, réservations et paramètres admin.

Structure des vues à créer

Dossier	Fichiers à créer
resources/views/layouts/	app.blade.php (layout principal)
resources/views/books/	index.blade.php, show.blade.php, create.blade.php, edit.blade.php
resources/views/loans/	index.blade.php, create.blade.php, retards.blade.php, mon-espace.blade.php
resources/views/reservations/	index.blade.php
resources/views/settings/	index.blade.php

Étape 1 — Layout principal (app.blade.php)

La navbar s'adapte automatiquement selon le rôle connecté :

```
@auth
    @if(auth()->user()->isLecteur())
        <a href="{{ route('lecteur.espace') }}">Mon espace</a>
    @endif
    @if(auth()->user()->isBibliothecaire() || auth()->user()->isAdmin())
        <a href="{{ route('loans.index') }}">Emprunts</a>
        <a href="{{ route('loans.retards') }}">Retards</a>
    @endif
    @if(auth()->user()->isAdmin())
        <a href="{{ route('settings.index') }}">Paramètres</a>
    @endif
@endauth
```

Étape 2 — books/show.blade.php (réponse UX n°1)

Affichage visuel des exemplaires par cases colorées :

```
@for($i = 0; $i < $book->total_copies; $i++)
    <div class="{{ $i < $book->available_copies
        ? 'bg-emerald-100 text-emerald-700'  // ✓ disponible
        : 'bg-gray-200 text-gray-400'         // ✗ emprunté
    }}">
```

```
        {{ $i < $book->available_copies ? '✓' : '✗' }}
    </div>
@endfor
// Texte clair : "1 exemplaire disponible sur 3 au total"
```

Étape 3 — loans/mon-espace.blade.php (réponse UX n°2)

Pénalités affichées en orange doux, sans être agressif :

```
@if($totalPenalites > 0)
    <div class="bg-amber-50 border border-amber-200 rounded-xl p-4">
        <p class="font-semibold text-amber-800">
            Des frais sont associés à votre compte
        </p>
        <p class="text-sm text-amber-700">
            Un total de {{ $totalPenalites }} FCFA correspond à des livres
            rendus en retard. Merci de vous rapprocher de la bibliothèque.
        </p>
    </div>
@endif
// PAS "Vous avez une amende" — ton bienveillant et solution proposée
```

Étape 4 — books/index.blade.php (réponse UX n°3)

Filtres dans l'ordre du parcours mental de l'utilisateur :

```
// 1. Recherche texte (titre, auteur) → l'utilisateur sait ce qu'il cherche
<input type="text" name="search" placeholder="Titre, auteur...">

// 2. Catégorie → explore par domaine
<select name="category">...</select>

// 3. Disponibles uniquement → filtre contextuel final
<input type="checkbox" name="disponible"> Disponibles uniquement
```

Étape 5 — Storage link pour les images

```
php artisan storage:link
```

Crée un lien symbolique pour que les couvertures uploadées soient accessibles.

✅ Tests Phase 5

Test	Compte	Action	Résultat attendu
1	admin	/books + /settings + /retards	Tout visible

Test	Compte	Action	Résultat attendu
2	biblio	/books/create + /loans	Visible | /settings → 403
3	lecteur	/books + /mon-espace	Visible | /books/create → 403
4	biblio	Ajouter livre + emprunter	Badge "1/2 dispo" dans catalogue
5	lecteur	Fiche livre épuisé → Réserver	"Vous êtes 1er dans la file"

🗀 Commit Git — Branche phase-5

```
git checkout -b phase-5
git add resources/views/
git commit -m "Phase 5 : toutes les vues Blade (layout, books, loans, reservations, settings)"
git push origin phase-5
```

Résumé Phase 5

Vue	Rôle
layouts/app.blade.php	Navbar dynamique selon le rôle
books/index.blade.php	Catalogue + recherche + 3 filtres
books/show.blade.php	Fiche + disponibilité visuelle + réservation
loans/index.blade.php	Liste emprunts avec badges retard
loans/retards.blade.php	Dashboard retards + pénalités
loans/mon-espace.blade.php	Espace lecteur bienveillant
reservations/index.blade.php	File d'attente
settings/index.blade.php	Paramètres admin

Phase 6 — Finalisation & Soutenance

> **Objectif**
> Créer les seeders avec données réalistes, écrire le README complet,
> merger toutes les branches sur main, et préparer les réponses aux 4 questions UX
> ainsi que les points techniques pour réussir la soutenance.

Étape 1 — Seeders réalistes

```
php artisan make:seeder BookSeeder
php artisan make:seeder LoanSeeder
```

BookSeeder : 8 livres réalistes (Roman, Informatique, Maths, Histoire...)

LoanSeeder : 1 emprunt en cours + 1 emprunt en retard + 1 historique avec pénalité + 1 réservation

Étape 2 — DatabaseSeeder (ordre important)

```
public function run(): void {
    $this->call([
        UserSeeder::class,     // 1. D'abord les users
        SettingSeeder::class,  // 2. Puis les paramètres
        BookSeeder::class,     // 3. Puis les livres
        LoanSeeder::class,     // 4. Enfin les emprunts/réservations
    ]);
}
```

Étape 3 — Reset complet et test final

```
php artisan migrate:fresh --seed
```

> **Que fait migrate:fresh ?**
> Supprime toutes les tables, les recrée proprement,
> puis lance tous les seeders dans l'ordre défini dans DatabaseSeeder.
> Utiliser avant la soutenance pour repartir sur une base propre.

✅ Tests Phase 6 — Checklist finale

✅	Test	Résultat attendu
☐	migrate:fresh --seed sans erreur	Toutes les migrations passent
☐	Catalogue → 8 livres affichés	BookSeeder fonctionnel
☐	Recherche "Laravel" → 1 résultat	Scope search() fonctionne
☐	Filtre "Informatique" → 3 livres	Filtre catégorie fonctionne

✅	Test	Résultat attendu
☐	/retards → 1 emprunt en retard	Scope overdue() fonctionne
☐	Mon espace lecteur → pénalité visible	LoanSeeder + vue bienveillante
☐	Admin → /settings → changer durée	SettingController fonctionne
☐	Emprunt → badge dispo décrémenté	Observer fonctionne
☐	Retour → badge dispo incrémenté	Observer retour fonctionne
☐	README présent sur GitHub	Documentation complète

🗁 Commit final — Merge sur main

```
git checkout -b phase-6
git add database/seeders/ README.md routes/web.php
git commit -m "Phase 6 : seeders réalistes + README + corrections finales"
git push origin phase-6

# Merge toutes les phases sur main
git checkout main
git merge phase-2
git merge phase-3
git merge phase-4
git merge phase-5
git merge phase-6
git push origin main
```

🎓 Réponses aux questions UX — Soutenance

Question 1 — Afficher 1 exemplaire disponible sur 3 ?

Ta réponse

Dans books/show.blade.php, j'affiche une rangée de cases colorées — une par exemplaire.
Cases vertes ✓ = disponibles, cases grises ✗ = empruntées.
Un texte clair dit "1 exemplaire disponible sur 3 au total".
Dans le catalogue, un badge "1/3 dispo" en vert ou "Indisponible" en rouge.
Justification : principe d'affordance visuelle — compris sans lire.

Question 2 — Communiquer une pénalité sans culpabiliser ?

Ta réponse

Dans mon-espace.blade.php, fond orange doux (amber) avec icône neutre 📋.
Texte : "Des frais sont associés à votre compte" — pas "Vous avez une amende".
Montant présenté avec une action claire : "Rapprochez-vous de la bibliothèque".
Justification : bonnes pratiques UX de bienveillance — informer sans sanctionner.

Question 3 — Filtres de recherche : ordre et justification ?

Ta réponse

1. Recherche texte (titre, auteur) → l'utilisateur sait déjà ce qu'il cherche.
2. Catégorie → pour explorer par domaine sans titre précis.
3. Disponibles uniquement → filtre contextuel final.

Justification : l'ordre suit le parcours mental naturel de l'utilisateur :
"je sais quoi → je cherche dans quel domaine → je vérifie si c'est dispo".

Question 4 — Un lecteur réserve un livre déjà réservé ?

Ta réponse

Dans ReservationController::store(), je vérifie qu'il n'a pas déjà une réservation active.
Si non, je crée sa réservation et j'affiche sa position : "Vous êtes 2e dans la file".
La file est ordonnée par reserved_at (premier arrivé, premier servi).
Au retour du livre, LoanController::retour() notifie le 1er en attente via Mailable.

🔑 Points techniques à maîtriser

Concept	Question probable	Ta réponse
hasMany vs belongsTo	Pourquoi hasMany sur Book ?	Un Book a plusieurs Loans → hasMany. La clé étrangère book_id est dans loans (le "Many" la porte)
Observer	Pourquoi pas dans le Controller ?	Séparation des responsabilités. Si Loan créé depuis n'importe où, available_copies est toujours mis à jour
Scope overdue()	Pourquoi un scope ?	Réutilisable, lisible, testable. Loan::overdue()->count() est expressif
Gate::before	Pourquoi Gate::before ?	Évite de vérifier isAdmin() dans chaque Policy. DRY (Don't Repeat Yourself)
Carbon	Comment calculer due_at ?	Carbon::now()->addDays($duree) — utilisation de Carbon pour toutes les dates

Résumé final du projet

Phase	Fichiers clés	Concept Laravel
1	.env, User.php, Breeze	Installation, Auth
2	Migrations, Modèles, Observer	Eloquent, Carbon
3	Middleware, Policies, Gate	Sécurité, Autorisation
4	4 Controllers, Mailable	Logique métier, Mail
5	10 vues Blade	Interface, UX
6	Seeders, README	Données de test, Doc

🎓 **Bonne soutenance !** Tu as tout ce qu'il faut pour décrocher une excellente note.